Agentized Graph Reasoning via Policy-Guided Deduction

Paper ID: YYY

ABSTRACT

This paper studies agentized graph reasoning, where deduction is conducted as a goal-directed sequence of controlled rule applications rather than exhaustive closure. The key challenge is selecting the next inference step. We introduce *policy-guided deduction*, in which a policy maps the current state and goal to a reasoning motif, to efficiently advance toward the target goal. We formalize the *policy selection problem* over Graph Association Rules, define a policy space, and propose a unified framework capturing facts, subgoals, proof frontier, budget and uncertainty. This formulation covers both optimal reasoning (minimizing proof cost) and anytime reasoning (maximizing utility under constraints). We show that policy optimization is intractable. Nonetheless, we develop effective policy strategies: deterministic, randomized, learned, and hybrid neuro-symbolic, guaranteeing soundness, goal-level completeness and approximation bounds. Using real-life datasets and workloads, we experimentally verify that policy-guided reasoning effectively reduces proof size and runtime, achieves near-optimal utility under tight budgets, and generalizes across diverse tasks.

1 INTRODUCTION

Rule-based graph reasoning is widely used in data-intensive applications such as anti-money laundering (AML), fraud detection, cybersecurity and risk analytics. In these domains, domain knowledge is encoded as rules (*e.g.,* Graph Association Rules, GARs [7]) that match graph patterns and derive new facts [8, 9], explanations [6, 16] or risks [18], by integrating logical reasoning with machine learning (ML). Given a graph and a rule set, classical reasoning systems apply rules to infer conclusions, typically via exhaustive mechanisms such as chase [7, 17], which compute the full logical closure.

Example 1: Consider an AML task over a financial transaction graph G. Vertices represent customers and accounts, while edges denote ownership and fund transfers. Suppose that an analyst asks: *Why is customer u flagged as high risk?* A typical explanation requires a reasoning chain such as: (a) u owns account a_1; (b) a_1 transfers a large amount to account a_2; (c) a_2 is linked to an identified high-risk entity; (d) risk propagates back to u. This chain provides a concise and understandable justification for the decision.

However, under classical rule-based reasoning, the system would not construct such a focused explanation. Instead, it would exhaustively apply all applicable rules on the graph, deriving a large number of intermediate facts, many of which are irrelevant to the query. For instance, it may explore unrelated transfer paths, expand multiple account clusters, and propagate risk signals across distant parts of G, incurring substantial reasoning cost and opaque outcome. □

This example motivates a paradigm shift in modern graph analytics. Reasoning is no longer by enumerating all logical consequences, but by constructing *goal-directed, decision-relevant derivations*. Analysts require concise, interpretable reasoning chains that justify target facts, often under strict latency or resource constraints. Consequently, reasoning must be selective, cost-aware, and aligned with the analytical objective, rather than exhaustive and task-agnostic.

The observations suggest that the key challenge in modern reasoning is not only *what* can be derived, but also *how* to derive it efficiently and purposefully. We need a mechanism to control the reasoning process, deciding which inference step to perform next in order to best support the target goal. That is, we model reasoning as a controlled, goal-directed and *agent-driven* process.

Limitations of prior approaches. Graph reasoning approaches [4, 5, 7, 11, 14, 17] fall short in data-intensive settings.

Not goal-directed. Chase-based reasoning is *exhaustive*, deriving all logical consequences regardless of a target query. In the AML example, this leads to excessive irrelevant transfer paths, misaligned with explaining why a specific customer is flagged.

Not cost-aware. Existing methods are *resource-oblivious*: they neither control proof size nor optimize latency, often recomputing redundant derivations, which is prohibitively costly in large graphs.

No control abstraction. There is no explicit mechanism to decide *which rule to apply next*. Rule application is either exhaustive or heuristic, lacking a decision layer that treats reasoning as a sequence of decisions and effectively supports its optimization.

Limited adaptability. Prior methods are static and task-agnostic, failing to adapt to varying goals, workloads or resource constraints.

Policy-based agentized deduction. The limitations suggest that the bottleneck is not expressiveness, but control: reasoning lacks a systematic, adaptive mechanism to guide inference. This motivates viewing reasoning as a *decision process*. We model reasoning as a sequence of decisions, each selecting the next inference step based on the current state and goal. Formally, we introduce a *policy*

$$\pi : (S, g) \rightarrow \mu,$$

which maps a deduction state S and a target goal g to a reasoning motif μ (*i.e.,* a rule application instance). Starting from an initial state S_0, the policy induces a *reasoning chain*

$$S_0 \xrightarrow{\mu_1} S_1 \xrightarrow{\mu_2} \cdots \xrightarrow{\mu_k} S_k.$$

Each step applies a selected motif to derive new facts toward g.

Beyond a fixed policy, modern reasoning requires *adaptive* and *context-aware control*. The next inference step depends on the state, goal, and resource constraints, calling for an *agent* that dynamically selects and adapts policies rather than following a static strategy. This agent-centric view supports learning, interaction with ML models, and adaptation to changing workloads.

This formulation offers several key advantages.

Decoupling semantics and control. It separates *what* can be derived (defined by GAR semantics and chase) from *how* it is derived (controlled by π). This decoupling preserves logical correctness while enabling goal-directed, cost-aware, and adaptive reasoning.

Unifying optimal and anytime reasoning. Modeling reasoning as a policy naturally subsumes different objectives: an *optimal policy* minimizes proof cost, yielding concise reasoning chains; and an *any-*

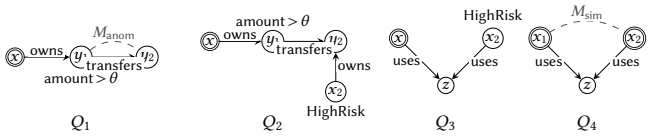


Figure 1: Patterns of the GARs of Example 2.

time policy maximizes decision utility under resource constraints by prioritizing high-impact steps. Thus, both minimal explanations and budgeted inference are captured within a unified framework.

Agentized deduction. We adopt *agentized* reasoning, in which inference is guided by a decision-making agent. The agent uses a policy π to select reasoning steps and adapts its behavior based on the current state, varying goals, workloads and resource constraints, to enable controllable and explainable reasoning at scale.

Problem statement. Based on this, we study *policy selection problem* for rule-based graph systems. Given a graph G, a set Σ of GARs, initial facts Γ_0, and a goal g, the task is to construct a reasoning chain that derives g. Specifically, the problem is to find a policy $\pi : (S, g) \rightarrow \mu$ that induces a chain of rule applications toward g, starting from the initial state $S_0 = (G, \Sigma, \Gamma_0)$, to optimize two objectives: (a) *optimal reasoning*, and (b) *anytime reasoning*.

Challenges. The problem poses several challenges.

Combinatorial search space. At each state, many rule instantiations are possible, yielding an exponential space of reasoning chains. We will show that the policy selection problems are intractable.

Non-local dependencies. The utility of a rule depends on downward derivations, making locally optimal choices globally suboptimal.

Soundness and completeness. Each step of the deduction must be sound under GAR semantics, *i.e.,* its preconditions hold and its consequence is entailed, which restricts feasible policy choices. At the same time, the policy should ensure goal-level completeness, *i.e.,* any entailed goal remains reachable under suitable policies.

Multi-objective tradeoffs. Policies must balance cost, utility and resource constraints, often under predefined tight budgets.

Contributions & Organization. We study policy selections for policy-guided reasoning, and make the following contributions.

(1) Foundations of agentized reasoning (Section 2). We revisit GAR-based reasoning and formalize *reasoning chains* and *motifs* as the building blocks of goal-directed inference. We explicitly decouple *semantics* (defined by GARs and chase) from *control*, providing a formal foundation for agent-driven reasoning.

(2) Policy selection as a unifying framework (Section 3). We formulate reasoning as a *policy selection problem* $\pi : (S, g) \rightarrow \mu$, define a rich state representation capturing facts, subgoals, frontier, budget and uncertainty, and unify *optimal reasoning* (cost minimization) and *anytime reasoning* (utility maximization under budget). We show that policy selection is NP-complete.

(3) Approximate policy-guided reasoning (Sections 4 and 5). We develop practical policy classes such as deterministic, randomized, learned and hybrid neuro-symbolic policies, together with execution and adaptation mechanisms. These provide a unified framework for *adaptive, budget-aware and workload-aware reasoning.*

(4) Theoretical guarantees (Section 6). We establish a theoretical characterization of policy-guided deduction, including (a) *soundness* and *goal-level completeness w.r.t.* chase, (b) *performance guarantees* for both optimal and anytime reasoning chains (approximation bounds), and (c) *termination, complexity and convergence* results.

(5) Experimental evaluation (Section 7). On real-world datasets and workloads, we show that policy-guided reasoning reduces proof size and latency, achieves near-optimal utility under tight budgets, and generalizes across diverse tasks. ! fill in experimental findings

Related and future work are in Sections 8 and 9; proofs are in [1].

Novelty. To our knowledge, this work is among the first to study policy selection for rule-based graph reasoning with budget and anytime semantics. It recasts reasoning from rule execution as *policy-driven deduction*, unifying logic, learning and control. By decoupling semantics from decision-making, it enables goal-directed, cost-aware and adaptive inference with guarantees, a scalable and explainable paradigm for graph reasoning at scale.

2 AGENTIZED GRAPH REASONING

This section reviews the foundations of rule-based graph reasoning. We consider GARs (Section 2.1), and chase as the semantic basis for deriving facts (Section 2.2). We then present reasoning chains for agentized goal-directed reasoning (Section 2.2). These notions characterize what can be derived under GAR semantics and policy-based control of inference (Section 2.3).

2.1 Graph Association Rules

We start with the primitives of graph reasoning.

Graphs and patterns. A *graph* is a labeled directed graph $G = (V, E, L, \mathcal{A})$, where V is a finite set of vertices, $E \subseteq V \times \Lambda_E \times V$ is a finite set of labeled edges, $L : V \rightarrow \Lambda_V$ assigns a label to each vertex, and $\mathcal{A}$ assigns to each vertex a finite tuple of attributes, without a fixed schema. A *pattern* is a graph $Q[\bar{x}] = (V_Q, E_Q, L_Q)$ whose nodes are the variables of a list $\bar{x}$, with labeled edges $E_Q \subseteq V_Q \times \Lambda_E \times V_Q$ and labels of Λ_V on nodes. A *match* of Q in G is a homomorphism $h : V_Q \rightarrow V$ that preserves labels and edges, *i.e.,* $L_Q(x) = L(h(x))$ for all $x \in V_Q$ and $(h(x), l, h(y)) \in E$ for every $(x, l, y) \in E_Q$.

Graph Association Rules. We start with predicates of GARs.

Predicates. A predicate over $Q[\bar{x}]$ is a Boolean condition defined on the attributes of vertices (variables) $\bar{x}$, including: (a) $x.A \oplus c$, where $x.A$ denotes the A attribute of vertex (variable) $x \in \bar{x}$, $\oplus \in \{=, \neq, <, >, \leq, \geq\}$ is a comparison operator and c is a constant; (b) $x.A \oplus y.B$, comparing attributes of two variables; (c) $l(x, y)$, indicating the existence of an edge with label l between $h(x)$ and $h(y)$; and (d) $M(x.\bar{A}, y.\bar{B})$, an ML predicate over attribute vectors $\bar{A}$ and $\bar{B}$ of x and y, respectively, such as a risk score exceeding a threshold.

GARs. A Graph Association Rule (GAR) is a rule of the form

$$\varphi = Q[\bar{x}](X(\bar{x}) \rightarrow p_0(\bar{x}_0)),$$

where $Q[\bar{x}]$ is a pattern, $X(\bar{x})$ is a finite set of predicates over $Q[\bar{x}]$, called the *preconditions* of ϕ, and $p_0(\bar{x}_0)$ is a predicate over $Q[\bar{x}]$, called the *consequence*, defined with a subset $\bar{x}_0 \subseteq \bar{x}$ of variables.

Given a match h of Q in G, the predicates of X are evaluated over

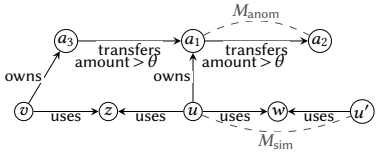


Figure 2: The graph G of Example 3; dashed curves are ML predicates.

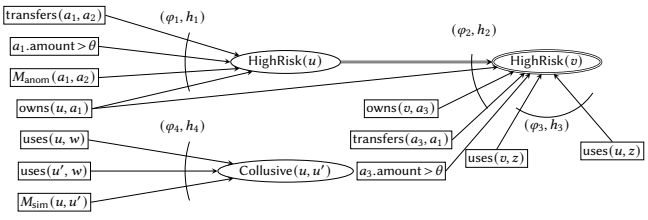


Figure 3: The chains of Example 4, drawn with shared facts merged: rectangles are facts of Γ_0, ellipses derived; an arc joins the premises of one application; the (φ_4, h_4) branch is a dead end, no path to g.

the matched vertices: attribute predicates compare the attribute values of $h(x)$ and $h(y)$, and an ML predicate $M(x.\bar{A}, y.\bar{B})$ evaluates the pre-trained model M on their attribute vectors.

If all predicates in X hold, the rule derives the consequent fact $p_0(h(\bar{x}_0))$. Thus, GARs combine structural pattern matching, attribute conditions, and ML prediction to derive new facts.

Example 2: The reasoning patterns behind the AML task of Example 1 are captured by GARs φ_1–φ_3 below; we also consider φ_4, which models cross-entity correlation. For brevity, we write $x.A = \text{true}$ as $A(x)$ when clear from context.

(1) Layered transfer risk propagation. Let $Q_1[x, y_1, y_2]$ contain edges $\text{owns}(x, y_1)$ and $\text{transfers}(y_1, y_2)$. Consider GAR

$\varphi_1 : Q_1[x, y_1, y_2]\big(y_1.\text{amount} > \theta \wedge M_{\text{anom}}(y_1, y_2) \rightarrow \text{HighRisk}(x)\big)$.

The rule states that a customer x is high-risk if its account y_1 makes a large anomalous transfer to y_2, where M_{anom} is a pretrained ML model for detecting suspicious behavior.

(2) Risk propagation. Let $Q_2[x, y_1, y_2, x_2]$ contain edges $\text{owns}(x, y_1)$, $\text{transfers}(y_1, y_2)$ and $\text{owns}(x_2, y_2)$. GAR φ_2 propagates risk along transfers: a customer x is high-risk if its account makes a large transfer to an account of a high-risk customer.

$\varphi_2 : Q_2[x, y_1, y_2, x_2]\big(y_1.\text{amount} > \theta \wedge \text{HighRisk}(x_2) \rightarrow \text{HighRisk}(x)\big)$.

Repeatedly applied, φ_2 propagates risk along transfer chains.

(3) Infrastructure sharing. Let $Q_3[x, x_2, z]$ contain edges $\text{uses}(x, z)$ and $\text{uses}(x_2, z)$. GAR φ_3 propagates risk through shared infrastructure: a customer x is high-risk if it shares infrastructure z with a high-risk customer x_2.

$$\varphi_3 : Q_3[x, x_2, z]\big(\text{HighRisk}(x_2) \rightarrow \text{HighRisk}(x)\big).$$

(4) Cross-entity correlation. Let $Q_4[x_1, x_2, z]$ contain edges $\text{uses}(x_1, z)$ and $\text{uses}(x_2, z)$. GAR φ_4 catches collusive entities that share infrastructure z and exhibit similar behavior under an ML predicate M_{sim}.

$\varphi_4 : Q_4[x_1, x_2, z]\big(x_1 \neq x_2 \wedge M_{\text{sim}}(x_1, x_2) \rightarrow \text{Collusive}(x_1, x_2)\big)$,

Their patterns are depicted in Figure 1: double circles mark the vertices on which a rule derives its consequent, and dashed curves are ML predicates. □

2.2 Chase-based Deduction

Chase. Let Γ_0 be a finite set of *validated facts* over G, *e.g.*, $\text{owns}(u, a)$, $a.\text{amount} > \theta$ or $M(a_1, a_2) = \text{true}$, collected from data sources. A rule φ is *applicable* on a fact set Γ under a match h of its pattern if all predicates of X hold under h *w.r.t.* Γ and the consequence $p_0(h(\bar{x}_0))$ is not yet in Γ; applying it derives the fact,

$$\Gamma \Rightarrow_{\varphi,h} \Gamma \cup \{p_0(h(\bar{x}_0))\}.$$

The *chase* repeatedly applies rules of Σ, $\Gamma_0 \Rightarrow \Gamma_1 \Rightarrow \cdots$, until a fixpoint, and returns the *closure* $\Gamma^* = \text{Chase}(G, \Sigma, \Gamma_0)$, all facts entailed by Σ over G. It terminates and is *confluent* (Church-Rosser, cf. [2]), so Γ^* is unique regardless of the order of applications; and the derived facts are logical consequences of Γ_0 and Σ, hence correct when those are [7].

Example 3: Continuing Example 2, consider a graph G with customers u, u', v, accounts a_1, a_2, a_3 and shared facilities z, w (Figure 2). Let Γ_0 consist of twelve facts: $\text{owns}(u, a_1)$, $\text{transfers}(a_1, a_2)$, $a_1.\text{amount} > \theta$, $M_{\text{anom}}(a_1, a_2)$, $\text{owns}(v, a_3)$, $\text{transfers}(a_3, a_1)$, $a_3.\text{amount} > \theta$, $\text{uses}(v, z)$, $\text{uses}(u, z)$, $\text{uses}(u, w)$, $\text{uses}(u', w)$ and $M_{\text{sim}}(u, u')$. The chase derives $\text{HighRisk}(u)$ via φ_1 under the match h_1 that maps x to u, y_1 to a_1 and y_2 to a_2, written $h_1 = \{x \mapsto u, y_1 \mapsto a_1, y_2 \mapsto a_2\}$: the edges $\text{owns}(u, a_1)$ and $\text{transfers}(a_1, a_2)$ match Q_1, and the conditions $a_1.\text{amount} > \theta$ and $M_{\text{anom}}(a_1, a_2)$ hold. It then derives $\text{HighRisk}(v)$ via φ_2 under $h_2 = \{x \mapsto v, y_1 \mapsto a_3, y_2 \mapsto a_1, x_2 \mapsto u\}$ and via φ_3 under $h_3 = \{x \mapsto v, x_2 \mapsto u, z \mapsto z\}$, both using the fact $\text{HighRisk}(u)$ just derived; and, independently of any goal, $\text{Collusive}(u, u')$ via φ_4. Nothing more is derivable: the closure is $\Gamma^* = \Gamma_0 \cup \{\text{HighRisk}(u), \text{HighRisk}(v), \text{Collusive}(u, u')\}$ in any order of the applications, and $\text{Collusive}(u, u')$ can never support $\text{HighRisk}(v)$. □

Limitations. Chase is exhaustive and task-agnostic: it derives the full closure regardless of the goal or the cost, and may generate many irrelevant intermediate facts. In practice, tasks call for *goal-directed* derivations without the full chase. We formalize such derivations as *reasoning chains*.

Reasoning chains. Given a goal g and initial facts Γ_0, a *reasoning chain* is a sequence that applies only rule instances relevant to g:

$$C = \langle(\varphi_1, h_1), \ldots, (\varphi_k, h_k)\rangle,$$

where each $\varphi_i \in \Sigma$ is applied under a match h_i to extend the current fact set, and the final step derives g, as opposed to chase.

Example 4: Continuing Examples 2 and 3, let $\Sigma = \{\varphi_1, \varphi_2, \varphi_3, \varphi_4\}$, Γ_0 be the twelve facts of Example 3, and $g = \text{HighRisk}(v)$. The chains toward g, drawn together in Figure 3, derive three facts on top of Γ_0: (φ_1, h_1) derives $\text{HighRisk}(u)$, and building on it, (φ_2, h_2) and (φ_3, h_3) derive the goal $\text{HighRisk}(v)$ in two ways, both resting on (φ_1, h_1), the only derivation of $\text{HighRisk}(u)$. Off the goal, (φ_4, h_4) derives $\text{Collusive}(u, u')$, a dead end: no rule takes Collusive as a premise, so no path from it reaches g. The fifteen facts at play are exactly the closure Γ^*. □

Reasoning chains give concise, goal-directed explanations.

2.3 From Semantics to Control

We have defined *what* can be derived under GAR semantics via chase and reasoning chains. We now turn to *how* to derive facts.

Semantics vs. control. Chase provides a complete and confluent characterization of all entailed facts, but it is exhaustive, task-agnostic and expensive. Reasoning chains restrict derivations to a goal, but they do not specify how to choose the next rule application. Thus, existing models capture semantics but lack a *control* mechanism.

Control. We model control via an *agent* that interacts with the rule system: given a state $S = (G, \Sigma, \Gamma)$ and a goal g, a *policy* $\pi : (S, g) \to \mu$ picks the next rule application, and starting from $S_0 = (G, \Sigma, \Gamma_0)$ it induces a reasoning chain. All such chains, bounded by the closure $\mathsf{Chase}(G, \Sigma, \Gamma_0)$, form the space that a policy searches. This formulation *decouples* semantics from control: GARs and chase define correctness (what is entailed), while the agent, via π, determines the inference process (how inference proceeds). As a result, we can design policies that are goal-directed, cost-aware and adaptive, without altering logical semantics.

3 THE POLICY SELECTION PROBLEM

This section formalizes policy-guided reasoning as an *agent-based decision model* over rule systems. We define the agent model (Section 3.1), and formulates policy selection as an optimization problem with cost and utility objectives (Section 3.2). We prove its intractability (Section 3.3).

3.1 Agent Model

We model policy-guided reasoning as an *agent* interacting with the rule system. Given a state $S = (G, \Sigma, \Gamma)$ and a target goal g, the agent selects a reasoning motif to extend the current state, via a policy $\pi : (S, g) \to \mu$, which serves as the agent's decision function.

Actions (motifs). We model actions as *motifs*, which are valid rule applications. Formally, a motif μ is a pair (φ, h), where $\varphi \in \Sigma$ is a GAR and h is a match of its pattern in G such that φ is applicable under h *w.r.t.* the current fact set Γ. Applying μ evaluates the preconditions of φ under h: if all hold, the consequent is derived and the state extends to $S' = (G, \Sigma, \Gamma \cup \{p_0(h(\bar{x}_0))\})$; otherwise the step fails, no fact is derived, and μ is marked failed and never retried. Write $\mathcal{A}(S)$ for the set of motifs applicable at state S, the candidates among which the policy chooses. Two kinds are kept out: those whose consequent $p_0(h(\bar{x}_0))$ is already in Γ, which would derive nothing new; and those that have failed before.

Agent dynamics. Starting from the initial state $S_0 = (G, \Sigma, \Gamma_0)$, the agent iteratively applies motifs, inducing a reasoning chain toward g. The agent records the outcome of every evaluation, and a failed motif is never retried.

State representation. For decision making, the state is represented by a *feature vector* $\phi(S, g, \mu)$ over candidate motifs μ, integrating logical with learned signals.

3.2 Problem Statement

We next formalize policy selection, starting with objectives.

Objective functions. For the objectives of policy selection, given an initial state $S_0 = (G, \Sigma, \Gamma_0)$ and a goal g, a policy π induces a reasoning chain $S_0 \xrightarrow{\mu_1} S_1 \xrightarrow{\mu_2} \cdots \xrightarrow{\mu_k} S_k$. Let $C(\pi; S_0, g)$ denote the induced chain. We evaluate π by a cost function over the chain and a utility function toward g under resource constraints.

Cost minimization (optimal reasoning). We want to have a policy that derives g with minimum cost. Let $\mathrm{cost}(\mu_i, S_{i-1}) \geq 0$ be the cost of applying motif μ_i at state S_{i-1}, charged for the evaluation of its preconditions whether they hold or not: applying (φ_4, h_4), *e.g.*,, pays for two evaluations, $u \neq u'$ and $M_{\mathsf{sim}}(u, u')$, even if one of them fails and nothing is derived, and define the total cost

$$\mathrm{cost}(C) = \sum_{i=1}^{k} \mathrm{cost}(\mu_i, S_{i-1}).$$

For randomized π, we minimize expected cost:

$$\min_{\pi} \ \mathbb{E}_{C\sim\pi}\big[\mathrm{cost}(C)\big] \quad \text{such that } g \in \Gamma_k.$$

This objective favors concise proofs (short, low-cost chains).

Utility maximization (anytime reasoning). Under a budget B (*e.g.*, time), we maximize the utility of the state. Let $\Gamma_{\leq B}$ be the fact set reached within budget, and $U(\Gamma_{\leq B}, g)$ measure progress toward g (*e.g.*, goal satisfaction, coverage of subgoals, or confidence). We solve

$$\max_{\pi} \ \mathbb{E}\big[U(\Gamma_{\leq B}, g)\big] \quad \text{such that } \mathrm{cost}(C) \leq B.$$

This prioritizes high-impact steps when full completion is infeasible.

Multi-objective optimization The two objectives combine into one, by weighing them against each other or by constraining one and optimizing the other. A common scalarization is

$$\max_{\pi} \ \mathbb{E}\big[U(\Gamma_{\leq B}, g) - \lambda\, \mathrm{cost}(C)\big],$$

for a tradeoff parameter $\lambda \geq 0$. In a constrained form, with a target utility η,

$$\min_{\pi} \ \mathbb{E}\big[\mathrm{cost}(C)\big] \ \text{ such that } \ \mathbb{E}\big[U(\Gamma_{\leq B}, g)\big] \geq \eta.$$

Both λ and η set how much cost a unit of utility is worth.

We now restate the scalarized objective per step. Both of its terms add up over the steps of a chain: the cost by the sum above, and the utility by summing what each step adds. Write S_μ for the state that applying μ at S leads to, with fact set Γ_μ, and $\Delta U(S, g, \mu) = U(\Gamma_\mu, g) - U(\Gamma, g)$ for the progress the move adds, its *marginal utility*. A chain from S then collects $\sum_i [\Delta U(S_{i-1}, g, \mu_i) - \lambda\, \mathrm{cost}(\mu_i, S_{i-1})]$. Let $V(S, g)$ be the best such total over chains from S; separating the first step from the rest,

$$V(S, g) = \max_{\mu} \ \big[\Delta U(S, g, \mu) - \lambda\, \mathrm{cost}(\mu, S) + V(S_\mu, g)\big],$$

where μ ranges over $\mathcal{A}(S)$, and V is zero when the budget runs out. At S_0 this is the scalarized objective above. The two objectives are special cases of the recursion. If utility is paid only upon deriving g, every chain to g earns the same utility, and maximizing $V(S, g)$ is minimizing the cost of reaching g. If the recursion stops when the budget runs out, it gathers utility within the budget, whether g is reached or not.

Policy selection problem. We next state the problem.

Optimal Policy Selection Problem (OPSM). Given an initial state $S_0 = (G, \Sigma, \Gamma_0)$ and a goal g, the problem is find a policy $\pi : (S, g) \to \mu$ that induces a reasoning chain C from S_0 to a state S_k with $g \in \Gamma_k$, such that the expected cost is minimized:

$$\min_{\pi} \ \mathbb{E}_{C\sim\pi}\big[\mathrm{cost}(C)\big] \quad \text{such that } g \in \Gamma_k.$$

Budgeted Policy Selection Problem (BOSP). Given S_0, g, and a budget B, find a policy π that maximizes the expected utility within budget:

$$\max_{\pi} \mathbb{E}\big[U(\Gamma_{\le B}, g)\big] \quad \text{such that } \text{cost}(C) \le B.$$

Example 5: Continuing Example 4, suppose two motifs are applicable: $\mu_1 = (\varphi_1, h_1)$, which derives HighRisk(u), one application from g via (φ_2, h_2); and $\mu_2 = (\varphi_4, h_4)$, which derives Collusive(u, u') and can never support g. Each of these rules has two preconditions; charge one unit per evaluation, so each application costs 2, and the chain through μ_1 and (φ_2, h_2) reaches g at cost 4.

(1) OPSM. An optimal policy derives g at the least cost: it applies (φ_1, h_1) and then (φ_2, h_2), a chain of cost 4, and never opens the branch of (φ_4, h_4).

(2) BOSP. With a budget $B = 4$ the same chain completes. With $B = 2$ no chain can complete, and a budgeted policy spends the step where it brings the most utility toward g: on (φ_1, h_1), whose consequence is one application from g, rather than on (φ_4, h_4). □

3.3 Intractability

We settle the complexity of policy selection under OPSM and BOSP.

The problems are nontrivial. At each state, many motifs may be applicable, and a policy must choose among them over multiple steps. This induces an exponentially large space of reasoning chains. Moreover, the effect of an action is non-local: a choice that appears promising may lead to dead ends or suboptimal cost later. Consequently, selecting an optimal sequence of actions has to search over all possible chains under global constraints, leading to intractability.

The decision version of OPSM is to decide, given (G, Σ, Γ_0, g) and a budget B, whether there exists a policy (equivalently, a reasoning chain) that derives g with $\text{cost}(C) \le B$. The decision version of BOSP is to decide, given (G, Σ, Γ_0, g), B and a threshold η, whether there exists a policy with $\text{cost}(C) \le B$ such that $U(\Gamma_{\le B}, g) \ge \eta$.

Theorem 1: *The decision versions of OPSM and BOSP are both* NP*-complete.* □

Proof sketch: Both problems are in NP: a certificate is a reasoning chain $C = \langle(\varphi_1, h_1), \ldots, (\varphi_k, h_k)\rangle$ of length bounded by B. We can check in polynomial time (PTIME) that each φ_i is applicable under h_i *w.r.t.* the current fact set, simulate the derived facts, and check whether g is derived (OPSM) or whether $U(\Gamma_{\le B}, g) \ge \eta$ (BOSP).

We show the lower bound of OPSM by reducing from NP-complete Set Cover (cf. [10]), where each set maps to a motif and deriving g requires covering all elements. NP-hardness of BOSP follows by reducing from NP-complete Knapsack (cf. [10]), where motifs have costs and utilities, and the budgeted policy must select a maximum-utility subset. So OPSM and BOSP are NP-complete. □

3.4 Solution Scheme

In light of the intractability (Theorem 1), we propose a 3-step scheme for *approximate* policy selection: given a goal g, rules Σ, a graph G, initial facts Γ_0 and a budget B, it returns the chain built within B, which derives g whenever the budget suffices.

Static analysis. The first step of the scheme analyzes the rules Σ and the goal g, once per goal and before any matching: it settles which predicates can lead to g at all, and how directly. The outcome should guides every step of the search.

Fact expansion. The second step expands the fact set, one derivation at a time, each applying a rule at a match of its pattern. Matching is incremental: each new fact of Γ is joined with the rule patterns it can take part in, on topology and labels alone, and every match it completes proposes a candidate motif (φ, h). A pattern may match many times, and not every match is valid: finding out takes evaluating the preconditions. The step proceeds in two modules, scoring and selection, as follows.

Scoring. The first module judges each candidate μ prior to its derivation. Whether μ actually derives is known only upon evaluating its preconditions; the module therefore estimates, at low cost, the two quantities that the objectives trade off: how much μ would advance the state toward g, and what its derivation would cost.

Selection and derivation. The second module selects a scored candidate μ via policy π, and derives μ by evaluating its preconditions: if the preconditions hold, then the consequent is added to Γ

Termination. The expansion repeats, charging each derivation against B, validated or not, and never deriving a candidate whose cost exceeds the remaining budget. When it stops depends on the objective. For optimal reasoning, deriving g does not end the search: it stops when no cheaper chain to g can still fit in what remains of B, and returns the cheapest chain found. For anytime reasoning, it stops when B runs out and returns the chain built. Either way, the chain returned costs at most B by construction.

4 SCORING MOTIFS

This section scores each candidate motif by two estimates, utility and cost: how much closer it brings the state to g (*relevance*), and how much resource it consumes (*cost*). We first identify what makes scoring hard, and then design the score to meet these difficulties.

Problem. Given a state $S = (G, \Sigma, \Gamma)$, a goal g, and a candidate motif $\mu = (\varphi, h)$, the task is to score μ , estimating how much applying μ serves the objectives: the utility the step adds toward g, and the cost it pays.

Challenges. Three challenges stand out.

[C1] Telling which applicable motifs lead toward the goal. At each step, many motifs are applicable: some carry the search straight toward g, some take detours, and some derive facts that can never reach g at all. The agent must be able to tell them apart.

[C2] Rules and matches are of different kinds. Judging a candidate asks two questions of different levels. Whether its rule can reach g at all is static, a question of the rule set; whether its premises are in place is dynamic, a question of the instance at hand. Each is easy to answer alone, the first settled once per goal by the rules and the second read off the state. The hard part is to answer both at once, at every step, in a single score, since the two are of different kinds: rules are symbolic conditions over variables, matches data-level assignments over G.

[C3] Scores must be fresh yet cheap. Each application derives new facts and shifts the evidence, so every candidate is re-scored at every step. Scoring thus has to be cheap: from scratch, *e.g.*, by grounding (NP-complete in the pattern), it would cost more than the deduction it guides.

What is needed, then, is guidance that is aware of the matches,

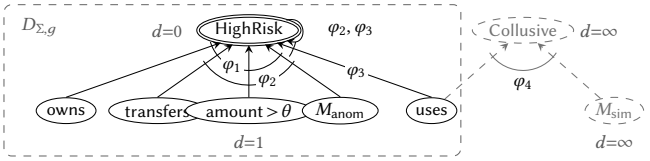


Figure 4: The dependency graph $D_{\Sigma,g}$ of Example 2; an arc marks a rule (AND), and $d(\cdot)$ the backward distance from HighRisk.

yet costs far less than computing them: settled once where the input is fixed, and maintained incrementally where it evolves.

4.1 The Scoring Function

In light of the challenges, we guide the exploration with a scoring method. The score is derived from the value $V(S, g)$. The best step at a state maximizes $\Delta U(S, g, \mu) - \lambda\,\text{cost}(\mu, S) + V(S_\mu, g)$. Since ΔU needs grounding and $V(S_\mu, g)$ the whole recursion, we estimate both instead. At each step, given a state $S = (G, \Sigma, \Gamma)$, the goal g and a candidate motif μ, the agent scores μ by

$$f(S, g, \mu) = \alpha \cdot \text{rel}(S, g, \mu) - \beta \cdot \text{cost}(\mu, S),$$

where $\text{rel}(S, g, \mu)$ estimates the utility side, $\text{cost}(\mu, S)$ the resource its derivation would consume, and $\alpha, \beta \geq 0$ balance the two, with β/α playing the role of λ.

The design of f meets the challenges: it combines a rule signal and a match signal (C1), encoded in one vector space (C2), the first settled once per goal and the second maintained incrementally, so that scoring a candidate is a lookup of $d(\cdot)$ and a dot product, far cheaper than grounding (C3).

4.1.1 Relevance. We measure the relevance of a motif from two sources. A candidate serves g only if its rule can lead to g and its premises are in place. The first is a question of the rules, settled once per goal; the second is a question of the instance at hand, and shifts as the chain grows. We thus instantiate rel by two signals, one from each source. From the rules, rel_{fam} reads a dependency graph of Σ, and tells whether a candidate can lead to g at all, and how directly. From the matches, rel_{evd} tests the facts at hand, encoded as vectors, and tells whether the premises of a candidate are actually in place. We develop the two in turn.

Scoring from the rules. The relevance of a candidate is first read off its rule: how directly the rule leads to g.

The dependency graph. We first organize the rules of Σ, once per goal, into a data structure that the score will consult, the *dependency graph* $D_{\Sigma,g}$ (Figure 4). It records for each predicate whether it can take part in deriving g at all, and through how few rule applications. $D_{\Sigma,g}$ has as nodes the predicates that can derive g, directly or indirectly, and an edge $q \to p$ whenever a rule φ uses q to derive p, labeled by φ; the same pair witnessed by several rules is kept once, and it carries the least weight when the edges are weighted for scoring (see below). Edges compose into paths: if $q \to q'$ and $q' \to q''$, then q reaches q'' by two rule applications. A path from q to g in $D_{\Sigma,g}$ is thus a chain of rule applications that derives g from q, and its length counts the applications. It is predicate-level, fixed by Σ and g alone, whatever the instance at hand.

Scoring. The static analysis prices each predicate by its distance to g. Let $d(q)$ be the fewest rule applications by which q can support g, *i.e.,* the length of the shortest path from q to g in $D_{\Sigma,g}$, with $d(q) = \infty$ if q is absent from $D_{\Sigma,g}$. (With edges weighted by a lower bound on the cost of one application, $d(q)$ lower-bounds the cost of supporting g instead; the policies will price completion by this reading.) The score of a candidate $\mu = (\varphi, h)$ sums these distances over the predicates of its rule φ:

$$\text{rel}_{\text{fam}}(\mu) = \gamma^{\sum_{q \in \varphi} d(q)},$$

where the sum ranges over the *distinct* predicates of φ: pattern edges, attribute and ML predicates and the consequent alike, each counted once however often it occurs; and $\gamma \in (0, 1)$ discounts each application still ahead. A rule serves g only when all its predicates do; one absent from $D_{\Sigma,g}$ scores the candidate 0, soft filtering.

$D_{\Sigma,g}$ and $d(\cdot)$ are built once per goal by a backward shortest-path computation over Σ, a BFS under unit cost (Appendix H); at runtime, the score of a candidate is a single lookup of $d(\cdot)$ per predicate.

Example 6: Take the rules of Example 2 and $g = \text{HighRisk}(v)$ (Figure 4). Under unit cost, the distances are $d = 0$ for HighRisk, and $d = 1$ for owns, transfers, amount $> \theta$, M_{anom} and uses, the premises of φ_1, φ_2 and φ_3. In contrast, Collusive and M_{sim} are absent from $D_{\Sigma,g}$ ($d = \infty$): they occur only in φ_4 and cannot reach HighRisk. (The self-loop on HighRisk is due to φ_2 and φ_3, which need a high-risk premise.) With $\gamma = 1/2$, a candidate of φ_3 scores $\gamma = 1/2$ (its distinct predicates uses and HighRisk at 1, 0), one of φ_2 scores $\gamma^3 = 1/8$ (owns, transfers, amount $> \theta$ and HighRisk at 1, 1, 1, 0; owns occurs twice in the pattern and counts once), one of φ_1 scores $\gamma^4 = 1/16$ (its four premises at 1 and the consequent at 0), and one of φ_4 scores 0 and is filtered out. The rules alone thus rank φ_3 above φ_2; Example 7 shows how the matches reverse this. □

Scoring from the matches. rel_{fam} alone cannot tell matches apart: it scores all matches of the same rule alike. We therefore score the match against the state, reading how much of a candidate's premises are already realized in Γ. This compares symbolic premises with data-level facts, two different kinds (C2). Hyperdimensional computing (HDC) [13] puts both in one space. It is training-free and incremental: no model is fit to data, and a new fact joins the encoding by one vector addition.

Hyperdimensional computing. We recall HDC [13]. It represents each symbol by a random bipolar vector of dimension *e.g.,* $D \approx 10^4$, $(+, -, +, -, \dots)$; with high probability, two distinct symbols are nearly orthogonal. Vectors are composed by *bundling* $\oplus$ (componentwise sum): the sum of several vectors remains similar to each summand, and dissimilar to any vector not in the sum. Positions are encoded by a fixed *permutation* ρ, a cyclic shift of coordinates: $\rho(v)$ and $\rho^2(v)$ are nearly orthogonal, even to v itself. Similarity is measured by the cosine.

Encoding facts. To place candidates in this space, we associate a random vector v_p with each predicate p, and a vector v_a with each vertex a that a match assigns to a variable of p. A fact $f = p(a_1, \dots, a_k)$, *i.e.,* predicate p under the assignment $a_1, \dots, a_k$ of a match, is represented by bundling, with the i-th assigned vertex shifted into its position: $v_f = v_p \oplus \bigoplus_{i=1}^{k} \rho^i(v_{a_i})$. The shift matters: it distinguishes *e.g.,* transfers(a, a') from transfers(a', a), which bundling alone would identify.

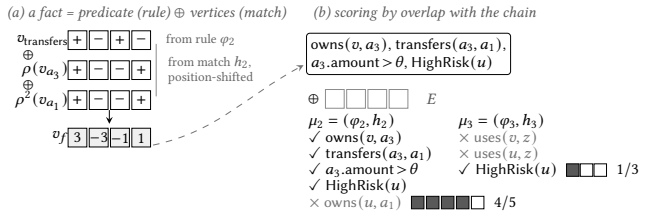


Figure 5: HDC scoring of Example 7: (a) facts bundle predicates and shifted vertices; (b) candidates score by cosine with E.

The evidence vector. To tell how much of a candidate is already in place, the facts established on the current reasoning chain are summarized in a single vector: they are bundled further into $E = \bigoplus_f v_f$, the *evidence vector*; E is maintained by one vector addition per new fact, and covers the chain built for g rather than the entire fact set Γ. The closer the bundled premises of a candidate are to E, the more of them are already established, and the less new computation the candidate demands (dimension (b)).

The score of a candidate is the distance between its grounded premises and the evidence vector:

$$\mathrm{rel}_{\mathrm{evd}}(\mu) = \cos\Big(\bigoplus_{p(\bar{x})\in\varphi} v_{p(h(\bar{x}))},\ E \Big).$$

By near-orthogonality, each predicate of a premise, and each vertex in its assigned position, that already occurs on the chain contributes $\approx +1$ to the inner product, and each absent one ≈ 0 (bounded in Appendix I). It is one dot product per candidate, in $O(D)$ time and free of matching; and as E grows at each step, the signal evolves with the state (cf. Section 5.4).

Putting the two together. The two signals combine into the relevance measure:

$$\mathrm{rel}(S, g, \mu) \;=\; \mathrm{rel}_{\mathrm{fam}}(\mu) \;+\; \nu \cdot \mathrm{rel}_{\mathrm{evd}}(\mu),$$

with $\nu \geq 0$ balancing the two. One signal is read off the fixed space, the other off the evolving state: $\mathrm{rel}_{\mathrm{fam}}$ is derived offline from the rule text, and $\mathrm{rel}_{\mathrm{evd}}$ is maintained online as the chain grows.

Example 7: Continue Example 6, with $D = 4$ for illustration (Figure 5). The symbols carry random vectors, *e.g.*, $v_{\mathrm{transfers}} = (+,-,+,-)$, $v_{a_3} = (+,+,-,-)$ and $v_{a_1} = (-,+,+,-)$, and ρ shifts each coordinate cyclically left by one. The fact $\mathrm{transfers}(a_3, a_1)$ is represented by $v_{\mathrm{transfers}} \oplus \rho(v_{a_3}) \oplus \rho^2(v_{a_1}) = (3,-3,-1,1)$. Suppose that, probing how to derive g, the chain has established $\mathrm{owns}(v, a_3)$, $\mathrm{transfers}(a_3, a_1)$, $a_3.\mathrm{amount} > \theta$ and $\mathrm{HighRisk}(u)$, the last derived by (φ_1, h_1); the evidence vector E is the sum of their representations. Consider $\mu_2 = (\varphi_2, h_2)$ and $\mu_3 = (\varphi_3, h_3)$, both deriving g (Figure 3). Scoring from the rules ranks μ_3 above μ_2 (γ against γ^3, Example 6). Scoring from the matches reverses the ranking: four of the five premises of μ_2 are summands of E, and only $\mathrm{owns}(u, a_1)$ is not yet established, while of the three premises of μ_3, only $\mathrm{HighRisk}(u)$ appears on the chain; the cosine of μ_2 with E is thus much higher. Hence $\mathrm{rel}(\mu_2) > \mathrm{rel}(\mu_3)$: the match signal favors continuing the chain built so far over starting a new branch. □

4.1.2 Cost. The second estimate, $\mathrm{cost}(\mu, S)$, prices the derivation of a candidate $\mu = (\varphi, h)$, *i.e.*, the evaluation of the preconditions of φ under h, estimated before any of them runs. The estimate ranks candidates; execution charges its actual cost against the budget, whether the preconditions hold or not.

The cost of a derivation. A derivation costs the sum of evaluating its preconditions:

$$\mathrm{cost}(\mu, S) = \sum_{p} T(p, h, S),$$

where p ranges over the preconditions of φ, and $T(p, h, S)$ estimates the cost of evaluating p under h, *e.g.*, the time the evaluation takes. When every candidate costs 1, the budget simply counts derivations.

Estimating predicate costs. The estimate $T(p, h, S)$ is read from a log when possible, and from a static measure otherwise. Execution keeps a log of predicate evaluations over the current and earlier chains: each record stores the observed cost of evaluating a motif μ, identified by its HDC vector v_p and v_a, and T is the average of these observations; a predicate whose result already stands in Γ costs a lookup only. Without records, T falls back to a static measure $h(p)$, which grows with the number of variables p reads and the size of their attribute vectors; it is scaled to the log by a factor δ, the average ratio of observed costs to static measures over the predicates logged so far, with a default before any log.

5 APPROXIMATING POLICY SELECTION

This section develops practical strategies to approximate policy selection: policies that follow the score, deterministic or randomized; policies that learn it from past reasoning chains; policies that guard it with logic; and execution that steers the search by the budget.

Problem. Given an initial state $S_0 = (G, \Sigma, \Gamma_0)$, a goal g, a budget B and the scoring function f, the task is to find and execute a policy π that, guided by f, selects a motif at each step so that its induced chain approaches the value $V(S_0, g)$: an optimal reasoning chain, deriving g at minimum cost, when the budget suffices, and an anytime reasoning chain, gathering the most utility toward g, when it does not. Exact selection is NP-complete (Theorem 1), so π must approximate, deciding each step by the score. Moreover, which case applies is settled only at runtime: π pursues an optimal chain while the remaining budget may still buy one, and switches to anytime reasoning once no chain can still reach g within it.

5.1 Deterministic and Randomized Policies

We start with policies that follow the score: deterministic ones commit to its top choice, and randomized ones sample accordingly.

Deterministic policies. A deterministic policy follows the score: at each state and goal, it evaluates the applicable motifs by their scores and chooses a best one. Over the reasoning chains, this is best-first search, greedy with respect to the score. Formally, it approximates OPSM by

$$\pi(S, g) = \arg\max_{\mu \in \mathcal{A}(S)} f(S, g, \mu),$$

where f is the shared scoring function.

At each step, incremental matching proposes candidate motifs (φ, h): the agent scores each by f, and derives a highest-scoring one, extending the chain when its preconditions all hold. The loop repeats until g is derived, no candidate remains, or the budget is exhausted. The policy commits to each choice: a branch that derives nothing still costs budget.

Randomized policies. The score is only an estimate, and committing to it makes its errors permanent: a branch the score underrates is never visited, and the same error recurs in every run. We develop randomized policies to improve the success rate, *i.e.,* the probability that a run derives g within the budget: the motif is drawn with probability increasing with the score,

$$\pi(S, g)(\mu) = \frac{\exp(f(S, g, \mu)/\tau)}{\sum_{\mu' \in \mathcal{A}(S)} \exp(f(S, g, \mu')/\tau)},$$

where f is the shared scoring function and $\tau > 0$ a configurable temperature: low τ approaches the deterministic policy, and high τ spreads the budget over branches.

Every reasoning chain keeps a nonzero probability, so a branch that would derive g is missed by every run with probability that shrinks exponentially in the number of runs.

Complexity. Both policies run the same loop: each step scores the m candidates in $O(m \cdot T_{\text{score}})$ and evaluates the selected one in T_{eval}, hence $O(B \cdot (m \cdot T_{\text{score}} + T_{\text{eval}}))$ over B steps; randomization adds only constant sampling overhead.

5.2 Learned Policies

We next develop learned policies to improve the score itself. The handcrafted score is fixed: relevance from rule text and premise overlap, cost from logged averages. Past reasoning chains carry what it misses: the rules settle which motifs *can* derive g, and the chains record which ones *did*, and at what cost. We therefore validate and correct the score against the chains.

Modeling and selection. A learned policy is parameterized by a model π_θ that scores each candidate motif and normalizes the scores into a distribution:

$$\pi_\theta(S, g)(\mu) = \text{softmax}\big(f_\theta(S, g, \mu)\big),$$

where f_θ is the learned score and π_θ the induced distribution over the applicable motifs. The function f_θ takes as input the features $\phi(S, g, \mu)$, which assemble the encodings of the candidate, the chain so far and the goal with what is read off the dependency graph $D_{\Sigma,g}$, as follows, and outputs a score for ranking μ.

Feature construction. The features of a candidate assemble what the state already computes. Per candidate $\mu = (\varphi, h)$, the input $\phi(S, g, \mu)$ to f_θ concatenates (1) the vector v_μ of the candidate, its premises and consequent instantiated by h and bundled by the HDC scheme; (2) the evidence vector E of the chain so far, maintained incrementally, and the vector v_g of the goal by the same scheme; (3) the distances of its premise predicates from g in $D_{\Sigma,g}$, as a profile $(s_0, s_1, \dots)$, s_k counting those k rule applications from g (Appendix H); and (4) scalars: $\text{rel}_{\text{fam}}(\mu)$, $\text{rel}_{\text{evd}}(\mu)$ and $\text{cost}(\mu, S)$, the number of premises already in Γ, the remaining budget, and the confidence of ML predicates.

The model f_θ is a two-layer perceptron over these features.

Learning. The model f_θ is trained offline and applied online. Offline, it learns from reasoning chains recorded from real runs, each labeled by execution itself with the cost it paid and the utility it gained; the chains are collected by the deterministic and randomized policies above. Online, at each step the agent encodes every applicable motif as $\phi(S, g, \mu)$, feeds it to the model, and ranks the candidates by its learned scores. We next train in two ways: supervised learning over the recorded choices, and reinforcement learning over the returns.

Supervised learning. Each recorded chain that derived its goal contributes one positive sample $(\phi(S_{t-1}, g, \mu_t), 1)$ per step, and each motif applicable at the same state but passed by a hard negative $(\phi(S_{t-1}, g, \mu^-), 0)$: it looks plausible, yet the chain passed it by [12]. Such negatives are noisy, since a motif passed by may derive g as well; the loss below asks only that the recorded choice outscore them, not that they be useless, and a choice confirmed by many chains is voted up. Nor are failed chains wasted: one that missed its goal still derived facts, and is relabeled as a positive trace for each of them [3]. The score is trained on the harvested pairs by a logistic ranking loss,

$$L_{\text{sup}} = \sum_{(\mu_t, \mu^-)} \log\big(1 + \exp\big(f_\theta(S_{t-1}, g, \mu^-) - f_\theta(S_{t-1}, g, \mu_t)\big)\big),$$

one term per recorded motif μ_t and a negative μ^- at the same state, large when the negative outscores the recorded choice and vanishing as the margin widens.

Reinforcement learning. The model is trained in episodes of self-play. In each episode, the policy plays out a reasoning chain, sampling an applicable motif by its current scores at each step, until it derives the goal or the budget runs out. The chain is then scored by its return $R(C)$, its utility minus its total cost, and each step has its probability pushed up or down in proportion to the return that followed it, by policy gradient [19], which maximizes the expected return over the chains the policy induces,

$$J(\theta) = \mathbb{E}_{C \sim \pi_\theta}\big[R(C)\big].$$

Episodes come from two sources. Recorded runs provide them for free, but price only the steps they took. Fresh episodes played offline price any candidate, taken before or not: apply it, run the policy to the end a constant number of times, and average the returns; the estimate sharpens with the number of runs. The updated model plays the next episodes, the exploration paid at training time; once trained, the model scores a candidate online by one forward pass. The price is credit assignment: one final return must be attributed to the steps of a long chain.

Example 8: Continuing Example 4, suppose a recorded run for goal HighRisk(v) applied (φ_1, h_1) and then (φ_2, h_2), at costs 2 and 3, and derived the goal, of utility 10. The two chosen motifs are positive samples, since the chain derived its goal through them, and (φ_4, h_4), applicable at the initial state but passed by, is a negative one. The chain is also a positive trace for goal HighRisk(u), derived on the way. The two steps also differ in value. The first led to a return of $10 - 2 - 3 = 5$ and the second to $10 - 3 = 7$, a grading that the binary labels do not carry. Fresh episodes price even the motifs passed by: applying (φ_4, h_4) and running the policy to the end a constant number of times, and averaging the returns, prices it as well, at the cost of extra deduction. A score learned from many such chains ranks motifs on productive routes high, even where the handcrafted f cannot tell them apart. □

Complexity. At each step, learned policies evaluate $f_\theta(S, g, \mu)$ for all $\mu \in \mathcal{A}(S)$, taking $O(|\mathcal{A}(S)| \cdot T_{\text{model}})$ time, where T_{model} is its inference cost. Over B steps, the total time is $O(B \cdot |\mathcal{A}(S)| \cdot T_{\text{model}})$.

Remark. Learned policies adapt to data and catch complex patterns, but require training data and may incur higher computational cost.

Algorithm HybNS
Input: graph G, rules Σ, initial facts Γ_0, goal g with $D_{\Sigma,g}$ and $d(\cdot)$, budget B, model f_θ.
Output: the optimal chain C^* and its cost $\hat{C}$, or an anytime chain when no chain within B exists.

1. $\hat{C} \leftarrow +\infty$; $C^* \leftarrow$ none; $F \leftarrow \{(\Gamma_0, 0, \emptyset)\}$;
2. **while** $F \neq \emptyset$ **do**
3. $(\Gamma, c, X) \leftarrow$ a node of F with the smallest $c + \min_\mu \text{price}(\mu)$;
4. $C \leftarrow$ motifs μ applicable at Γ and not in X, with $d(p_0) < \infty$, deriving new facts, $c + \text{price}(\mu) \le B$ and $c + \text{price}(\mu) < \hat{C}$;
5. **if** $C = \emptyset$ discard the node and continue;
6. score C by f_θ, keep the top-k, and re-rank by price, ties by rel;
7. apply the best μ, charging its actual cost c_μ;
8. **if** some precondition fails, push $(\Gamma, c + c_\mu, X \cup \{\mu\})$ to F;
9. **else**
10. $\Gamma' \leftarrow \Gamma \cup \{p_0(h(\bar{x}_0))\}$; $c' \leftarrow c + c_\mu$;
11. **if** $g \in \Gamma'$: $C^* \leftarrow$ the chain to Γ'; $\hat{C} \leftarrow \min(\hat{C}, c')$;
12. **else** push (Γ', c', X) to F;
13. push $(\Gamma, c, X \cup \{\mu\})$ to F;
14. **if** C^* is none, build an anytime chain: greedily apply admissible best-rel motifs deriving new facts while the budget remains;
15. **return** C^* and $\hat{C}$ (and the anytime chain when C^* is none).

Figure 6: Hybrid neuro-symbolic reasoning as branch and bound; $\text{price}(\mu)$ lower-bounds completing the chain through μ (Appendix H).

5.3 Hybrid Neuro-Symbolic Policies

We develop hybrid policies to guard the learned score with logic. A hybrid policy rules out the provably useless by logic, and ranks the rest by the model.

Modeling and selection. Formally, a hybrid policy maximizes the learned score over the admissible candidates,

$$\pi(S, g) = \arg \max_{\mu \in \mathcal{A}(S) \cap C(S)} f_\theta(S, g, \mu),$$

where $C(S)$ denotes the logically admissible motifs: it keeps those that (a) can reach g, *i.e.,* $d(p_0) < \infty$, and (b) derive something new, *i.e.,* the consequent fact is absent from Γ; applicability comes for free from matching.

Symbolic filtering. The filter drops every rule whose consequent cannot reach g, and nothing else, since $d(p_0) = \infty$ if and only if no chain of rules leads from p_0 to g; deduction thus stays complete (Corollary 4). This is the hard form of the soft filtering of rel_{fam}, over the same $d(\cdot)$. The check is a property of the rule, not of the match, settled once per goal by one lookup of $d(\cdot)$ per rule, and rules that cannot reach g are never matched at all. The survivors are then ranked by the model, since $d(p_0) < \infty$ promises no grounded derivation in G.

Complexity. Once per goal, the check is one lookup of $d(\cdot)$ per rule; per step, the policy costs exactly a learned one, over a candidate set whose matching has already shrunk to the rules that can reach g.

5.4 Execution and Adaptation

We next study how the agent executes and adapts policies to approximate optimal and anytime reasoning in practice. Execution unifies two complementary behaviors: (a) constructing *anytime reasoning chains* that maximize utility under a limited budget, and (b) approximating *optimal reasoning chains* when resources suffice. The budget steers the search between the two, and which applies is revealed only as the search proceeds.

Algorithm. The algorithm unifies anytime and optimal reasoning: under a limited budget, it yields high-utility partial chains, and with sufficient budget, it refines them into optimal reasoning chains. It is a best-first branch and bound search (Figure 6), in two regimes.

(1) Optimal reasoning. The search keeps a set of open branches, and the best chain found so far, C^* with its cost $\hat{C}$ (line 1). At each round it expands the branch that looks cheapest to complete, by the cost so far plus the least completion price of its candidates (line 3). Of the branch's candidates it keeps only those that can still beat C^* within the budget (lines 4-5), scores them by f_θ, and applies the best (lines 6-7). A failed application marks the motif and returns the state to the frontier (line 8); a successful one pushes the new state, and the untried alternatives with it (lines 12-13). Reaching g records the chain as C^* and tightens $\hat{C}$ (line 11), and the loop runs until no branch survives (line 2), so C^* is then optimal.

(2) Anytime reasoning. If no chain was found, the bounds have certified that none can fit the budget; a greedy pass then applies an admissible motif of the highest relevance at each step, while the budget remains (line 14).

Branch and bound. Why does the search end with the optimal chain, or with a proof that none fits the budget? The two bounds do the work. The completion price lower-bounds the cost of every chain through a candidate, since $d(p_0)$ lower-bounds deriving g from its consequent; a branch cut by it thus completes only over budget or without improving on the recorded chain, and nothing that could win is ever cut. Expanding cheapest first brings the first chain, and with it $\hat{C}$, early, and each chain recorded only tightens the pruning; with exact costs this is A*, and each test is one lookup of $d(\cdot)$.

Constrained re-ranking. Re-ranking serves anytime reasoning. When the search proves that no chain can reach g within the budget, the agent changes objective, from reaching g at the least cost to collecting the most utility before the budget runs out. A single greedy pass then applies, at each state,

$$\pi(S, g) = \arg \max_{\mu \in \mathcal{A}(S) \cap C(S)} \text{rel}(S, g, \mu),$$

while the budget remains (line 14). Greedy suffices here, since with no completion to price, a step earns only its own marginal utility. The criterion is the utility term of the score: with the budget fixed, cost turns from a criterion into a constraint, and what is left to maximize at each step is its marginal utility toward g. Marginal utility, however, is known only after the step is applied, so the pass ranks candidates by its estimate, rel: rel_{fam} tells how much the derived fact would be worth toward g, the closer the more; and rel_{evd} tells whether the premises are in place, and hence whether the step derives at all, since a failed step earns nothing.

Example 9: Continue Example 4: the bank now escalates high-risk customers to a blacklist, via a rule φ_5 from $\text{HighRisk}(x)$ to $\text{Blacklist}(x)$; Γ_0 also carries the pattern of u for another customer w; and the goal is $g = \text{Blacklist}(v)$. Hence $d(\text{Blacklist}) = 0$, $d(\text{HighRisk}) = 1$, and Collusive cannot reach g in $D_{\Sigma,g}$. With

a budget of two steps, the model prefers (φ_4, h_4), but the filter drops it by a lookup: Collusive cannot reach Blacklist. Its next pick (φ_1, h_1) prices at $1 + d(\text{HighRisk}) = 2$, exactly the budget; it fires and derives HighRisk(u). With one step spent, every remaining candidate completes at $1 + 2 = 3$, over budget; all branches are cut and no chain is recorded, certifying that g is unreachable in two steps. A greedy pass by relevance then spends the last step on (φ_2, h_2), whose premise HighRisk(u) just landed on the chain, deriving HighRisk(v) and stopping one short of g with an informative partial chain. With a budget of five, the first step goes as before. The three candidates then price at two alike, and relevance breaks the tie: (φ_2, h_2), whose premise already stands on the chain, is expanded first, deriving HighRisk(v), and φ_5 closes at the next step, a chain of three steps; the recorded chain lands at $\hat{C} = 3$. Every remaining branch prices at three, no improvement possible, and is cut at one lookup each; the three-step chain is certified optimal, and the line for w is never opened. □

Regimes over time. The regime needs no explicit scheduling: the search closes while a completion may still fit, and crosses to banking when the bounds certify that none can. Early steps are guided mainly by rel_{fam} while the evidence is still sparse, and rel_{evd} strengthens as the chain grows.

Complexity. Each expansion scores the candidates of one branch in $O(|\mathcal{A}(S)| \cdot T_{\text{model}})$, applies one in T_{eval}, and prunes by lookups of $d(\cdot)$. The number of expansions is bounded by the number of distinct partial chains, finite but potentially large; the recorded chain prunes it in practice, and the best-first order finds the first chain after few expansions.

Adaptation across workloads. The symbolic layer transfers exactly: a new goal asks for one backward BFS to rebuild $D_{\Sigma,g}$ and $d(\cdot)$, and a new graph for rematching. The learned score is retrained on chains of the new workload, possibly with lightweight adaptation of its weights. The weights α, β and the temperature τ can be adjusted online, and the cost log and the evidence vector adapt by themselves as chains grow.

6 THEORETICAL GUARANTEES

This section analyzes the theoretical properties of policy-guided deduction. These properties certify the design of Sections 4 and 5: the shared score f guides the search without corrupting it, and the strategies of Section 5 inherit their guarantees from the score they share. We establish (a) *correctness guarantees*, showing that reasoning remains sound and complete *w.r.t.* GAR semantics (Section 6.1), (b) *performance guarantees*, characterizing the quality of constructed reasoning chains relative to optimal and budgeted objectives (Section 6.2), and (c) *complexity and convergence*, providing bounds on termination and computational cost (Section 6.3).

6.1 Correctness Guarantees

We study both the soundness and completeness.

Soundness. We first show that policy-guided deduction preserves the logical correctness of GAR semantics (the chase).

Theorem 2: *Let Γ_0 be a set of valid initial facts over G, and Σ a set of* GARs. *For any policy π, every fact derived along a reasoning chain induced by π is logically entailed by (G, Σ, Γ_0). That is, for any state $S_k = (G, \Sigma, \Gamma_k)$ reached by π, $\Gamma_k \subseteq \Gamma^* = \text{Chase}(G, \Sigma, \Gamma_0)$.* □

Proof sketch: Each step applies a motif $\mu = (\varphi, h)$ only when GAR φ is applicable under match h, hence derives a fact entailed by the current facts and Σ. Since chase enumerates all such valid applications, every derived fact must be in Γ^*. Thus, $\Gamma_k \subseteq \Gamma^*$. □

Theorem 2 ensures that policy-guided deduction never returns spurious facts, at any step, regardless of how the policy selects actions; this holds for every policy class, and in particular for anytime reasoning under budget constraints.

Goal-level Completeness. We next study whether policy-guided deduction can derive a target goal whenever it is logically entailed.

Theorem 3: *Let $\Gamma^* = \text{Chase}(G, \Sigma, \Gamma_0)$ be the chase closure. If a goal $g \in \Gamma^*$, then there exists a policy π such that the reasoning chain induced by π derives g. Moreover, if π is* fair, *i.e., it does not indefinitely ignore applicable motifs that contribute to deriving g, then π derives g whenever $g \in \Gamma^*$.* □

Proof sketch: Since $g \in \Gamma^*$, there exists a finite chase sequence that derives g from Γ_0. Following this sequence yields a policy that derives g. For a fair policy, any motif in such a derivation is eventually selected. Since the sequence is finite, all required motifs are applied in finite steps, and g is derived. □

Soundness and completeness together ensure that deduction derives only correct facts and can reach any entailed goal.

Corollary 4: *Filtering by the dependency graph preserves completeness: a policy that never selects motifs whose consequent p_0 has $d(p_0) = \infty$ still derives g whenever $g \in \Gamma^*$, if it is fair over the surviving motifs.* □

Proof sketch: If $g \in \Gamma^*$, some chase sequence derives g; on it, every consequent reaches g through the suffix, *i.e.*, $d(p_0) < \infty$, so the whole sequence survives the filter, and Theorem 3 applies over the survivors. □

6.2 Performance Guarantees

We next study the quality of reasoning chains produced by policy-guided deduction relative to optimal and budgeted objectives.

Approximation for optimal reasoning chains. Let C^* be an optimal reasoning chain that derives g with minimum cost, and C_π the chain induced by a policy π. We study how close C_π is to C^*.

Theorem 5: *When the scoring function $f(S, g, \mu)$ is* consistent *with cost reduction, i.e., it prioritizes motifs that lead to lower-cost completions, the reasoning chain C_π induced by a greedy policy satisfies*

$$\text{cost}(C_\pi) \leq \kappa \cdot \text{cost}(C^*),$$

for some constant $\kappa \geq 1$ depending on the quality of f. □

Proof sketch: By consistency, each greedy choice approximates the best step toward a minimum-cost completion. Thus the greedy sequence incurs at most a constant-factor loss per step relative to the optimum. Aggregating on the chain has $\text{cost}(C_\pi) \leq \kappa \cdot \text{cost}(C^*)$. □

As a result, simple deterministic policies can get near-optimal reasoning chains when guided by appropriate scoring functions.

When the branch and bound search of Figure 6 is used, the guarantee strengthens from approximation to exactness.

Theorem 6: *Let C^* and $\hat{C}$ be the chain and its cost returned by the search of Figure 6. Then C^* derives g at cost $\hat{C}$, and when no branch survives, no chain deriving g within budget B costs less than $\hat{C}$: C^* is then optimal within B.* □

Proof sketch: A branch is cut only when the cost so far, the candidate, and the bound $d(\cdot)$ together exceed B or reach $\hat{C}$. Since $d(\cdot)$ lower-bounds the cost of every completion (Appendix H), every chain through a cut branch costs more than B or no less than $\hat{C}$. The search ends only when every branch is cut or completed, so no cheaper chain to g within B is missed. □

The relevance measure rel meets this consistency requirement. Indeed, $\mathrm{rel}_{\mathrm{fam}}(\mu) = \gamma^{\sum_{q\in\varphi} d(q)}$ is strictly decreasing in $d(\cdot)$, which is an admissible lower bound on the remaining cost, *i.e.,* the remaining proof length under unit cost; hence greedy selection guided by f descends this lower bound at each step.

Guarantees for anytime reasoning chains. We next analyze the quality of partial reasoning under a budget B; note that utility accumulates, since steps only add facts and U is monotone.

Theorem 7: *Assume that U is monotone and submodular. Then the utility achieved by a greedy policy under budget B satisfies*

$$U(C_\pi) \geq (1 - 1/e) \cdot U(C^*),$$

where C^* is the optimal chain under budget B. □

Proof sketch: Since U is monotone and submodular, marginal gains diminish as Γ grows. The greedy policy selects the motif with the largest marginal gain at each step, which yields a $(1 - 1/e)$-approximation under a cardinality (budget) constraint [15]. □

The utilities used in this paper (Section 3.2) meet these assumptions: goal satisfaction and subgoal coverage are coverage functions over the fact set, and are hence monotone and submodular; confidence-based variants are monotone by definition. Thus the $(1-1/e)$-approximation applies to the budgeted policies of Section 5 when instantiated with these utilities.

6.3 Convergence and Bounded Cost

We also study whether policy-guided deduction terminates.

Theorem 8: *For a finite graph G and a finite rule set Σ, any reasoning chain induced by a policy π terminates in finitely many steps; under a budget B, every chain it returns moreover costs at most B.* □

Proof sketch: Each step either adds a new fact to Γ or retires a candidate, recording its failure and never retrying it; since the derivable facts and the matches of Σ in G are finite, the process terminates in finitely many steps. Under a budget, execution applies a motif only when its cost keeps the chain within B, and halts when no branch survives, so each chain returned is bounded by B by construction. □

Policy-guided deduction thus terminates and stays within budget; its runtime and its convergence to the chase closure under fair exploration are bounded in the appendix (Theorems 9 and 10).

7 EXPERIMENTAL STUDY

Experimental setting.

Datasets.

Workloads.

Baselines.

Rule libraries and motif construction.

Evaluation metrics.

Default parameters.

Configuration.

Experimental findings.

Exp-1: Overall performance comparison.

Policy types.

State features.

Motif selection strategies.

Soundness and completeness.

Exp-2: Anytime performance and budget analysis

Exp-3: Approximation to optimal chains

Exp-4: Generalization across workloads

Exp-5: Scalability analysis

Exp-6: Ablation study.

Remove fact context, subgoals, proof frontier, budget, and uncertainty one by one to show why $\phi(S, g, \mu)$ matters.

Summary. We find the following. (1) (2) (3) (4)

8 RELATED WORK

We categorize the related work as follows.

Rule-based graph reasoning.

Goal-directed deduction and planning.

Reusable motif reasoning.

Neuro-symbolic reasoning.

9 CONCLUSION

We have proposed a policy-guided deduction framework for agentized graph reasoning, supporting goal-directed and cost-aware inference. By abstracting policy selection, we have unified optimal and anytime reasoning, and developed practical policy classes with adaptive execution mechanisms. We have established correctness, performance, and convergence guarantees, providing a solid foundation for controlled reasoning.

Topics for future work include: (a) *learning optimal policies*: developing learning methods (*e.g.,* reinforcement or imitation learning) with stronger guarantees on optimality and generalization; (b) *multi-agent reasoning*: supporting collaborative or competitive agents for distributed and large-scale graph reasoning; and (c) *richer constraints and uncertainty*: incorporating probabilistic rules and soft constraints for robust reasoning under uncertainty.

REFERENCES

[1] 2026. Full version. *link*.

[2] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases*. Addison-Wesley.

[3] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. 2017. Hindsight Experience Replay. In *NeurIPS*. 5048–5058.

[4] Wenchao Bai, Wenfei Fan, Daji Li, Jian Li, Shuhao Liu, Mingliang Ouyang, and Qiang Yuan. 2025. MiniClean: A Single-Machine System for Cleaning Big Graphs. In *SIGMOD (demo)*.

[5] Wenchao Bai, Wenfei Fan, Shuhao Liu, Xiaoke Zhu, and Jiahui Jin. 2025. Rule-Based Graph Cleaning with GPUs on a Single Machine. In *SIGMOD*.

[6] Wenfei Fan, Lihang Fan, Dandan Lin, and Min Xie. 2025. Explaining GNN-based Recommendations in Logic. *PVLDB* 18, 3 (2025), 715–728.

[7] Wenfei Fan, Ruochun Jin, Muyang Liu, Ping Lu, Chao Tian, and Jingren Zhou. 2020. Capturing Associations in Graphs. *PVLDB* 13, 11 (2020), 1863–1876.

[8] Wenfei Fan, Yang Leng, Daji Li, Shuhao Liu, Mingliang Ouyang, Yaoshu Wang, Min Xie, and Qiang Yuan. 2025. DreamCreek: AI for Battery Formation and Grading. In *ICDE*. IEEE, 4544–4547.

[9] Wenfei Fan, Daji Li, Peiyu Liang, Shuhao Liu, Yaoshu Wang, Yiming Wang, Min Xie, and Runjie Zhang. 2024. Graph Association Analyses for Early Drug Discovery. *PVLDB* 17, 12 (2024), 4293–4296.

[10] Michael Garey and David Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company.

[11] Floris Geerts, Giansalvatore Mecca, Paolo Papotti, and Donatello Santoro. 2013. The LLUNATIC Data-Cleaning Framework. *PVLDB* 6, 9 (2013), 625–636.

[12] Jan Jakubův and Josef Urban. 2017. ENIGMA: Efficient Learning-based Inference Guiding Machine. In *CICM*. 292–302.

[13] Pentti Kanerva. 2009. Hyperdimensional Computing: An Introduction to Computing in Distributed Representation with High-Dimensional Random Vectors. *Cognitive Computation* 1, 2 (2009), 139–159.

[14] Zuhair Khayyat, Ihab F. Ilyas, Alekh Jindal, Samuel Madden, Mourad Ouzzani, Paolo Papotti, Jorge-Arnulfo Quiané-Ruiz, Nan Tang, and Si Yin. 2015. BigDansing: A System for Big Data Cleansing. In *SIGMOD*. 1215–1230.

[15] George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. 1978. An analysis of approximations for maximizing submodular set functions - I. *Math. Program.* 14, 1 (1978), 265–294.

[16] Kehan Pang, Wenfei Fan, Min Xie, and Dandan Lin. 2026. Explaining GNN Negatives Globally and Locally. In *ICDE*.

[17] Fereidoon Sadri and Jeffrey D. Ullman. 1980. The Interaction between Functional Dependencies and Template Dependencies. In *SIGMOD*.

[18] Shenzhen Institute of Computing Sciences. 2022. Fishing Fort. *https://en.sics.ac.cn/col84/index*.

[19] Richard S. Sutton, David McAllester, Satinder Singh, and Yishay Mansour. 2000. Policy Gradient Methods for Reinforcement Learning with Function Approximation. In *NeurIPS*. 1057–1063.

A PROOF OF THEOREM 1

We first show that both problems admit polynomial-size certificates that can be verified in polynomial time (PTIME). A certificate is a reasoning chain $C = \langle(\varphi_1, h_1), \ldots, (\varphi_k, h_k)\rangle$ of length $k \le B$.

Upper bounds. We guess a reasoning chain starting from Γ_0. At each step, we check that: (a) φ_i is applicable under h_i *w.r.t.* the current fact set (predicate evaluation is polynomial in the size of G and Γ), and (b) the derived fact is correctly added. This yields Γ_k (or $\Gamma_{\le B}$) in $O(B \cdot T_{\text{apply}})$ time. Then we verify the following.

- OPSM: We verify that $g \in \Gamma_k$ and $\text{cost}(C) \le B$. Since $k \le B$ and each step is polynomial, verification is polynomial.
- BOSP. We additionally compute $U(\Gamma_{\le B}, g)$. For standard utilities used in this paper (e.g., goal satisfaction, subgoal coverage, or sums of per-fact scores), U is computable in polynomial time from $\Gamma_{\le B}$. We then check $U(\Gamma_{\le B}, g) \ge \tau$ and $\text{cost}(C) \le B$.

Thus, both problems are in NP.

Lower bounds. We reduce from classical NP-complete problems.

OPSM. We reduce from NP-complete Set Cover (cf. [10]). Given a universe $\mathcal{U} = \{e_1, \ldots, e_n\}$ and a collection $\mathcal{S} = \{S_1, \ldots, S_m\}$, we construct a graph and GARs such that each set S_i corresponds to a motif μ_i that "covers" elements in S_i by deriving corresponding facts. The goal g is satisfied iff all elements in $\mathcal{U}$ are covered. Each motif incurs unit cost. Thus, finding a minimum-cost reasoning chain that derives g corresponds to selecting a minimum set cover.

BOSP. We reduce from NP-complete knapsack (cf. [10]). Each motif μ_i has a cost (weight) and contributes utility (value) toward achieving g. Given a budget B, maximizing $U(\Gamma_{\le B}, g)$ corresponds to selecting a subset of motifs with maximum total utility under a cost constraint.

In both cases, the constructions are in PTIME, and solutions correspond to valid reasoning chains. Hence, OPSM and BOSP are NP-hard. Combined with upper bounds, they are NP-complete. □

B PROOF OF THEOREM 2

Each step in a reasoning chain applies a motif $\mu = (\varphi, h)$ only when φ is applicable under h *w.r.t.* the current fact set. By GAR semantics, such an application derives a fact that is logically implied by existing facts and Σ. Since the chase computes the closure of all such valid applications, every fact derived by the policy must also appear in Γ^*. Thus, policy-guided deduction derives only entailed facts. □

C PROOF OF THEOREM 3

Since $g \in \Gamma^*$, there exists a chase sequence that derives g from Γ_0. We construct a policy that, at each step, selects the next rule application in this sequence. This policy induces a reasoning chain that exactly follows the chase derivation and reaches g. Thus, any entailed goal can be derived by a suitable policy.

Under fairness, any motif that is part of a valid derivation of g will eventually be selected. Since the chase sequence that derives g consists of finitely many such motifs, the policy will eventually apply all of them. Thus, g is derived in finite steps. □

D PROOF OF THEOREM 5

The greedy policy selects motifs that locally minimize cost or maximize progress toward the goal. Each selection approximates the best available choice in the optimal chain. By the study [15] of submodular or monotone progress, the accumulated deviation from optimality is bounded by a constant factor κ.

It remains to verify that the relevance measure rel satisfies the consistency condition, *i.e.,* it prioritizes motifs that lead to lower-cost completions. We show this for rel_{fam}; rel_{evd} is a soft prior isolated by λ and does not enter the guarantee.

(1) *Admissibility.* For each predicate q, the distance $d(q)$ computed by backward BFS over the dependency graph $D_{\Sigma,g}$ is at most the number of rule applications of any derivation of g that passes through q, by the shortest-path property of BFS. Hence $d(\cdot)$ is an admissible lower bound on the remaining proof length.

(2) *Ranking.* Since $\gamma \in (0, 1)$, the score $\text{rel}_{\text{fam}}(\mu) = \gamma^{\sum_{q\in\varphi} d(q)}$ is strictly decreasing in $\sum_{q\in\varphi} d(q)$; hence ranking candidates by rel_{fam} coincides with ranking by the summed distances, *i.e.,* each greedy choice follows the admissible lower bound of the remaining cost. Composing the per-step deviations over the chain yields the constant κ, which depends on γ and the level gap. □

E PROOF OF THEOREM 7

Under submodularity, the marginal gain of each motif decreases as more facts are derived. The greedy policy selects motifs with the largest marginal gain, which yields a $(1 - 1/e)$-approximation by submodular maximization under budget constraints [15]. □

F PROOF OF THEOREM 8

One can verify that the number of distinct ground facts that can be derived is finite (bounded by $|V|$, $|\Sigma|$, and the attribute domains). Each step adds a new fact to Γ, and no fact is added twice. Hence, the reasoning process can perform at most $|\Gamma^*|$ steps, where Γ^* is the chase closure, and must terminate. □

G BOUNDED COST AND CONVERGENCE

Theorem 9: *Let $\mathcal{A}(S)$ be the set of applicable motifs at state S, and let T_{score} denote the cost of evaluating a motif. Then the total runtime of policy-guided deduction over B steps is $O(B \cdot |\mathcal{A}(S)| \cdot T_{score})$.* □

Proof sketch: Each step evaluates all $\mu \in \mathcal{A}(S)$, costing $O(|\mathcal{A}(S)| \cdot T_{\text{score}})$ time. Over B steps, this yields $O(B \cdot |\mathcal{A}(S)| \cdot T_{\text{score}})$. □

Theorem 10: *If execution continues without a budget constraint and the policy eventually explores all applicable motifs (i.e., is fair), then the resulting fact set converges to the chase closure Γ^*.* □

Proof sketch: By fairness, every applicable motif is eventually applied, hence all facts in Γ^* are derived. Since derivations are sound, no extra facts are produced, so the limit equals Γ^*. Termination implies convergence to this fixed point. □

H ALGORITHMS FOR THE DEPENDENCY GRAPH

Algorithm BuildDG (Figure 7) constructs the dependency graph $D_{\Sigma,g}$ and the distance table $d(\cdot)$, by a backward BFS from the goal over the rules; when rule edges are weighted, the BFS becomes a Dijkstra computation, relaxing $d[q] \leftarrow \min(d[q], d[p] + w(\varphi))$ at

Algorithm BuildDG
Input: goal g, rule set Σ.
Output: $D_{\Sigma,g} = (N, A)$ and table d.

1. $N \leftarrow \{\text{pred}(g)\}$, $A \leftarrow \emptyset$, $d[\text{pred}(g)] \leftarrow 0$; enqueue $\text{pred}(g)$ into queue Q;
2. **while** Q is not empty **do**
3. dequeue p from Q;
4. **for each** rule $\varphi \in \Sigma$ with consequent p **do**
5. **for each** predicate q in the pattern or preconditions of φ **do**
6. add $q \to p$ to A;
7. **if** $q \notin N$ **then** $N \leftarrow N \cup \{q\}$, $d[q] \leftarrow d[p] + 1$, enqueue q into Q;
8. **return** $D_{\Sigma,g} = (N, A)$ and d.

Figure 7: Building $D_{\Sigma,g}$ and $d(\cdot)$.

Algorithm ScoreF
Input: $D_{\Sigma,g} = (N, A)$ with table d, rule φ of the candidate, decay factor $\gamma \in (0, 1)$.
Output: $\text{rel}_{\text{fam}}(\mu)$.

1. $s \leftarrow 0$;
2. **for each** predicate q of φ (pattern, preconditions and consequent) **do**
3. **if** $q \notin N$ **then return** 0; /* *outside $D_{\Sigma,g}$: filtered out* */
4. $s \leftarrow s + d[q]$;
5. **return** γ^s.

Figure 8: Scoring a candidate from $D_{\Sigma,g}$.

line 7, where $w(\varphi)$ is a lower bound on the cost of one application of φ (Section 4.1.2). It visits each rule once per predicate of the rule, *i.e.,* in $O(|\Sigma|)$ time. The table also bounds completion: any chain through a candidate with consequent p_0 completes at cost at least $d(p_0)$, since each application on the way costs at least its edge weight and the path is shortest. Algorithm ScoreF (Figure 8) scores a candidate motif $\mu = (\varphi, h)$ from the table. The match h plays no role here: rel_{fam} is a property of the rule, and the match enters only in rel_{evd}.

The learned policies read the graph through a vector $e_D = \bigoplus_k \rho^k(b_k)$, where b_k bundles the vectors of the predicates k rule applications from g; shifted onto their own channels, the depths never interfere, $v_q, \rho(v_q), \rho^2(v_q)$ being mutually orthogonal, and a predicate that cannot reach g never enters e_D. The vector is settled in the same backward pass as $d(\cdot)$, once per goal; for Example 6, $e_D = v_{\text{HighRisk}} \oplus \rho(b_1)$, with b_1 bundling owns, transfers, amount $> \theta$, M_{anom} and uses. A candidate bundles its premise predicates, each once, into $v_\varphi = \bigoplus_{q \in \varphi} v_q$ and probes e_D per depth: the cosine $s_k = \cos(\rho^k(v_\varphi), e_D)$ tells how many of them sit k applications from g, the shift ρ^k singling out the depth-k part of e_D. The probes give a profile $(s_0, s_1, \dots)$ per candidate, *e.g.,* proportional to $(s_0, s_1) = (1, 4), (1, 3), (1, 1)$ for candidates of φ_1, φ_2, φ_3 of Example 6, and $(0, 1)$ for φ_4, whose other premises sit on no channel. The profile is fed whole to f_θ; the handcrafted rel_{fam} reads only its fixed decay $\gamma^{\sum_k k s_k}$, *e.g.,* the $\gamma^4, \gamma^3, \gamma$ of Example 6.

I ERROR OF COSINE READINGS

We justify the counts read from cosines in Section 4: the reading error is worst-case, data-free, and shrinks as the dimension grows. Each symbol carries a vector of D independent coordinates, uniform in $\{-1/\sqrt{D}, +1/\sqrt{D}\}$, hence of unit norm, and distinct symbols are independent.

Lemma 11: *Let E bundle n facts and n_q count the summands in which symbol q occurs. For $n \le \sqrt{D}$, with probability at least $1 - \delta$ over the random vectors, $|\cos(v_q, E) - n_q/\sqrt{n}| \le \sqrt{2\ln(2/\delta)/D}\,(1 + o(1))$.* □

Proof sketch. Write $\cos(v_q, E) = \langle v_q, E\rangle / \|E\|$. The inner product splits into a signal n_q and a noise $\eta_q = \sum_{f_i \neq q} \langle v_q, v_{f_i}\rangle$. Each $\langle v_q, v_{f_i}\rangle$ is a sum of D independent terms of $\pm 1/D$, so η_q sums nD such terms, and Hoeffding's inequality gives $|\eta_q| \le \sqrt{2n\ln(2/\delta)/D}$ with probability $1 - \delta$. The norm concentrates as well: the cross terms of $\|E\|^2$ sum to $O(n/\sqrt{D})$, so $\|E\| = \sqrt{n}\,(1 + o(1))$ for $n \le \sqrt{D}$. Dividing by $\|E\|$ yields the bound. □

The bound prices the reading. Adjacent counts differ by $1/\sqrt{n}$, and the noise is $\sqrt{2\ln(2/\delta)/D}$; their ratio is $\sqrt{2n\ln(2/\delta)/D}$, a third for a chain of a hundred facts at $D = 10^4$ and $\delta = 0.01$, and far less for the few premises of one candidate. An absent symbol has $n_q = 0$, hence $|\cos| \le 0.033$, the "≈ 0" of Section 4; the same bound covers the probes s_k on the dependency encoding e_D, with the premises of a candidate in place of n. The guarantee is worst-case over the random vectors, needs no data, and tightens as D grows; a learned embedding reads nothing of what its scores count.